

Chapter 4 — Sliding Window

0. What You Will Learn in This Chapter

Sliding Window is one of the most important patterns for array and string problems. It is used when the problem asks about a **contiguous subarray** or **contiguous substring** and you need to find:

- maximum
- minimum
- longest
- shortest
- count
- fixed-size result
- at most K condition
- exactly K condition
- frequency match
- permutation/anagram window

By the end of this chapter, you should understand:

- What a sliding window is
- Why sliding window avoids repeated work
- Fixed-size vs variable-size windows
- Longest valid window template
- Shortest valid window template
- Frequency map windows
- Counting windows
- At most K vs exactly K
- When sliding window fails
- How sliding window differs from two pointers
- How to handle edge cases
- How to explain sliding window in interviews

Core LeetCode problems covered in this chapter:

- 1.1. Maximum Average Subarray I
 - 2.2. Maximum Number of Vowels in a Substring of Given Length
 - 3.3. Longest Substring Without Repeating Characters
 - 4.4. Minimum Size Subarray Sum
 - 5.5. Permutation in String
 - 6.6. Find All Anagrams in a String
 - 7.7. Minimum Window Substring
 - 8.8. Longest Repeating Character Replacement
 - 9.9. Max Consecutive Ones III
 - 10.10. Fruit Into Baskets
 - 11.11. Subarray Product Less Than K
 - 12.12. Count Number of Nice Subarrays
 - 13.13. Binary Subarrays With Sum
 - 14.14. Subarrays with K Different Integers
 - 15.15. Number of Substrings Containing All Three Characters
-

1. What Is Sliding Window?

A window is a contiguous part of an array or string.

Example:

```
nums = {1, 2, 3, 4, 5}
```

Some windows:

```
{1, 2}  
{2, 3, 4}  
{3, 4, 5}
```

In strings:

```
s = "abcabcbb"
```

Some substrings/windows:

```
"abc"  
"bca"  
"cab"
```

A sliding window uses two pointers:

```
left  
right
```

The current window is usually:

```
[left ... right]
```

When right moves forward, the window expands.

When left moves forward, the window shrinks.

2. Why Sliding Window Exists

Many subarray and substring problems have a brute force solution:

```
for every start:  
    for every end:  
        check subarray/string from start to end
```

This is usually $O(n^2)$, and if checking each window takes time, it may become $O(n^3)$.
Sliding window avoids recomputing information from scratch.

Example:

For a fixed-size sum window:

```
nums = {1, 2, 3, 4}  
k = 2
```

Windows:

```
{1,2} sum = 3  
{2,3} sum = 5  
{3,4} sum = 7
```

Instead of recomputing each sum:

```
newSum = oldSum - outgoingElement + incomingElement
```

This makes each step $O(1)$.

The key idea:

Add the new right element and remove the old left element.

3. Core Need / Why Use Sliding Window Over Other Patterns?

Sliding Window is useful when the problem is about a **contiguous segment** and the window can be updated efficiently.

3.1 Why Not Basic Traversal?

Basic traversal tracks simple state for individual elements.

Sliding window tracks a range.

Example:

Longest Substring Without Repeating Characters.

You need to know all characters currently inside the substring.

One variable is not enough.

3.2 Why Not Hashing Alone?

Hashing can tell you frequency/existence.

But sliding window controls which characters are currently active.

Hashing is often used **inside** sliding window.

Example:

```
unordered_map<char, int> freq;
```

This map only represents the current window, not the whole string.

3.3 Why Not Prefix Sum?

Prefix sum is excellent for static range sums and exact subarray sum with negatives.

Sliding window is better when:

- window size changes dynamically
- all numbers are positive
- you need longest/shortest contiguous segment
- you can shrink while preserving logic

Important:

For exact sum with negative numbers, prefix sum + hash map is usually better.

Example:

Subarray Sum Equals K with negative numbers.

3.4 Why Not Two Pointers?

Sliding window is a type of two-pointer technique, but it maintains state for a whole window.

Two pointers often compare boundaries or pairs.

Sliding window maintains:

- sum
- product
- frequency map

- distinct count
 - zero count
 - matched characters
-

4. When to Use Sliding Window

Use sliding window when you see these clues.

4.1 Contiguous Subarray or Substring

Problem says:

- subarray
- substring
- contiguous
- consecutive
- window

Examples:

- Longest Substring Without Repeating Characters
- Minimum Size Subarray Sum
- Maximum Average Subarray I

Important distinction:

Subarray	contiguous part of array
Substring	contiguous part of string
Subsequence	not necessarily contiguous

Sliding window is for subarrays/substrings, not general subsequences.

4.2 Fixed Window Size

Problem says:

- size k
- length k
- exactly k consecutive elements
- substring of length k

Examples:

- Maximum Average Subarray I
 - Maximum Number of Vowels in a Substring of Given Length
 - Permutation in String
 - Find All Anagrams in a String
-

4.3 Longest Valid Window

Problem says:

- longest substring without...
- longest subarray with at most...
- maximum consecutive...
- at most K replacements/flips

Examples:

- Longest Substring Without Repeating Characters
 - Longest Repeating Character Replacement
 - Max Consecutive Ones III
 - Fruit Into Baskets
-

4.4 Shortest Valid Window

Problem says:

- minimum size
- shortest subarray
- minimum window
- smallest substring containing...

Examples:

- Minimum Size Subarray Sum
 - Minimum Window Substring
-

4.5 Frequency Matching

Problem says:

- permutation in string
- anagram
- contains all characters
- same frequency
- window with required characters

Examples:

- Permutation in String
 - Find All Anagrams in a String
 - Minimum Window Substring
-

4.6 Counting Valid Windows

Problem says:

- number of subarrays
- count substrings
- at most K
- exactly K
- product less than K

Examples:

- Subarray Product Less Than K
 - Count Number of Nice Subarrays
 - Binary Subarrays With Sum
 - Subarrays with K Different Integers
 - Number of Substrings Containing All Three Characters
-

5. When NOT to Use Sliding Window

5.1 Negative Numbers With Exact Sum

Sliding window often depends on monotonic behavior.

For positive numbers:

- adding right increases sum
- removing left decreases sum

With negative numbers, this breaks.

Example:

```
nums = {1, -1, 1}
k = 1
```

Adding -1 decreases sum.

Removing -1 increases sum.

For exact sum with negative numbers, use prefix sum + hash map.

5.2 Problem Is About Subsequences

Example:

Longest Increasing Subsequence.

Subsequence is not contiguous.

Use DP or binary search.

5.3 Cannot Update Window Efficiently

If removing left or adding right is expensive, basic sliding window may not work.

You may need:

- monotonic deque
- heap
- multiset
- segment tree

Examples:

- Sliding Window Maximum
 - Sliding Window Median
-

5.4 Predicate Is Not Window-Monotonic

For variable sliding window, there must be a clear way to shrink until valid or expand until valid.

If validity changes unpredictably, another pattern may be needed.

6. Two Main Types of Sliding Window

Type 1 — Fixed-Size Sliding Window

Window size is exactly k .

Example:

```
Find max sum of subarray of length k.
```

The window always has size k.

Process:

2. Add right element.
 3. If size exceeds k, remove left element.
 4. When size equals k, update answer.
-

Fixed Window Template

```
int left = 0;
int windowSum = 0;
int best = INT_MIN;

for (int right = 0; right < nums.size(); right++) {
    windowSum += nums[right];

    if (right - left + 1 > k) {
        windowSum -= nums[left];
        left++;
    }

    if (right - left + 1 == k) {
        best = max(best, windowSum);
    }
}
```

Type 2 — Variable-Size Sliding Window

Window size changes depending on validity.

Example:

Longest substring without repeating characters.

Process:

5. Expand right.
 6. Add current element to window state.
 7. While window is invalid, shrink left.
 8. Update answer.
-

Longest Valid Window Template

```
int left = 0;
int best = 0;

for (int right = 0; right < nums.size(); right++) {
```

```
// add nums[right]

while (/* window invalid */) {
    // remove nums[left]
    left++;
}

best = max(best, right - left + 1);
}
```

Shortest Valid Window Template

```
int left = 0;
int best = INT_MAX;

for (int right = 0; right < nums.size(); right++) {
    // add nums[right]

    while (/* window valid */) {
        best = min(best, right - left + 1);

        // remove nums[left]
        left++;
    }
}

return best == INT_MAX ? 0 : best;
```

7. Mental Models

7.1 Window as a Backpack

The window contains current active elements.

When right moves:

put new item into backpack

When left moves:

remove old item from backpack

The map/count/sum tells you what is currently inside.

7.2 Expand Until Bad, Then Shrink Until Good

Used for longest valid window.

Example:

Longest substring without repeating characters.

expand right
if duplicate appears, shrink left until duplicate removed
update best

7.3 Expand Until Good, Then Shrink While Good

Used for shortest valid window.

Example:

Minimum Window Substring.

expand right until window contains all required chars
then shrink left while still valid
update minimum

7.4 Every Element Enters Once and Leaves Once

This is why sliding window is $O(n)$.

- right moves from 0 to $n - 1$
- left also moves from 0 to $n - 1$ at most once

Even with nested while, total pointer movement is $O(n)$.

7.5 Window Validity

Every sliding window problem depends on defining:

What makes the current window valid?

Examples:

Longest no-repeat substring	no character frequency > 1
Max Consecutive Ones III	zero count $\leq k$
Fruit Into Baskets	distinct fruit count ≤ 2
Minimum Window Substring	all required chars satisfied
Subarray Product Less Than K	product $< k$

8. Pattern Recognition Checklist

Ask:

9. Is the problem about a contiguous subarray or substring?
10. Is there a fixed window size?
11. Is it asking for longest valid window?
12. Is it asking for shortest valid window?
13. Is it asking to count valid windows?
14. Does the condition involve frequency?
15. Does it say "at most K"?

16. Does it say “exactly K”?
17. Are all numbers positive for sum/product problems?
18. Can I add right and remove left efficiently?

If yes, sliding window is likely.

9. C++ Tools Needed

9.1 Window Length

For inclusive window [left ... right]:

```
int length = right - left + 1;
```

9.2 Frequency Array

For lowercase English letters:

```
vector<int> freq(26, 0);
```

For uppercase English letters:

```
vector<int> freq(26, 0);
```

For ASCII:

```
vector<int> freq(128, 0);
```

9.3 Hash Map Frequency

For general values:

```
unordered_map<int, int> freq;  
unordered_map<char, int> freq;
```

9.4 Erasing Zero Counts

For distinct-count windows:

```
freq[x]--;  
  
if (freq[x] == 0) {  
    freq.erase(x);  
}
```

Without erasing, freq.size() is wrong.

9.5 max() and min()

```
best = max(best, right - left + 1);  
best = min(best, right - left + 1);
```

9.6 long long

For products and large sums:

```
long long product = 1;
long long sum = 0;
```

10. Core Templates

Template 1 — Fixed-Size Sum Window

```
int windowSum = 0;
int best = INT_MIN;

for (int right = 0; right < nums.size(); right++) {
    windowSum += nums[right];

    if (right >= k) {
        windowSum -= nums[right - k];
    }

    if (right >= k - 1) {
        best = max(best, windowSum);
    }
}
```

Template 2 — Fixed-Size Window With Left Pointer

```
int left = 0;
int windowSum = 0;
int best = INT_MIN;

for (int right = 0; right < nums.size(); right++) {
    windowSum += nums[right];

    if (right - left + 1 > k) {
        windowSum -= nums[left];
        left++;
    }

    if (right - left + 1 == k) {
        best = max(best, windowSum);
    }
}
```

Template 3 — Longest Valid Window

```
int left = 0;
int best = 0;

for (int right = 0; right < nums.size(); right++) {
    // add nums[right]

    while (/* window invalid */) {
        // remove nums[left]
        left++;
    }

    best = max(best, right - left + 1);
}

return best;
```

Template 4 — Shortest Valid Window

```
int left = 0;
int best = INT_MAX;

for (int right = 0; right < nums.size(); right++) {
    // add nums[right]

    while (/* window valid */) {
        best = min(best, right - left + 1);

        // remove nums[left]
        left++;
    }
}

return best == INT_MAX ? 0 : best;
```

Template 5 — Frequency Window for Strings

```
vector<int> freq(128, 0);

int left = 0;
int best = 0;
```

```

for (int right = 0; right < s.size(); right++) {
    freq[s[right]]++;

    while (/* invalid frequency condition */) {
        freq[s[left]]--;
        left++;
    }

    best = max(best, right - left + 1);
}

```

Template 6 — At Most K Distinct

```

unordered_map<int, int> freq;

int left = 0;
int best = 0;

for (int right = 0; right < nums.size(); right++) {
    freq[nums[right]]++;

    while (freq.size() > k) {
        freq[nums[left]]--;

        if (freq[nums[left]] == 0) {
            freq.erase(nums[left]);
        }

        left++;
    }

    best = max(best, right - left + 1);
}

```

Template 7 — Count Valid Windows

```

int left = 0;
int answer = 0;

for (int right = 0; right < nums.size(); right++) {
    // add nums[right]

    while (/* window invalid */) {

```

```

        // remove nums[left]
        left++;
    }

    answer += right - left + 1;
}

return answer;

```

Why add right - left + 1?

If [left ... right] is valid, then every subarray ending at right and starting from any index between left and right is valid.

Template 8 — Exactly K Using At Most

Many problems asking “exactly K” can be transformed:

```
exactly K = atMost(K) - atMost(K - 1)
```

Template:

```

int exactlyK(vector<int>& nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}

```

Used for:

- Count Number of Nice Subarrays
- Binary Subarrays With Sum
- Subarrays with K Different Integers

11. Major Variations of Sliding Window

Fixed-size sum	fixed	Maximum Average Subarray
Fixed-size frequency	fixed	Permutation in String
Longest valid	variable	Longest Substring Without Repeating
Shortest valid	variable	Minimum Window Substring
At most K	variable	Fruit Into Baskets
Exactly K	atMost trick	Nice Subarrays
Count valid windows	counting	Product Less Than K

Variation	Window Type	Example
Frequency satisfaction	variable	Minimum Window Substring
Zero/flip count	variable	Max Consecutive Ones III

12. Full Problem Walkthrough 1 — LeetCode 643: Maximum Average Subarray I

12.1 Problem Summary

Given array `nums` and integer `k`, find the maximum average of any contiguous subarray of length `k`.

Example:

`nums = {1, 12, -5, -6, 50, 3}`

`k = 4`

Output:

12.75

Best window:

{12, -5, -6, 50}

Sum = 51, average = 12.75.

12.2 How to Identify This Pattern

Problem says:

- subarray
- length `k`
- maximum average

This is fixed-size sliding window.

12.3 Brute Force Idea

For each starting index, compute sum of next `k` values.

Time: $O(n * k)$

12.4 Optimized Intuition

Maintain sum of current window.

When window slides:

`newSum = oldSum + incoming - outgoing`

12.5 C++ Code

```
class Solution {
public:
    double findMaxAverage(vector<int>& nums, int k) {
        int windowSum = 0;

        for (int i = 0; i < k; i++) {
            windowSum += nums[i];
        }

        int bestSum = windowSum;

        for (int right = k; right < nums.size(); right++) {
            windowSum += nums[right];
            windowSum -= nums[right - k];

            bestSum = max(bestSum, windowSum);
        }

        return (double)bestSum / k;
    }
};
```

12.6 Dry Run

nums = {1, 12, -5, -6, 50, 3}
k = 4

First window:

{1,12,-5,-6}
sum = 2

4	50	1	{12,-5,-6,50}	51	51
5	3	12	{-5,-6,50,3}	42	51

Return:

51 / 4 = 12.75

12.7 Complexity

Time: O(n)

Space: O(1)

12.8 Edge Cases

- $k = 1$
 - k equals array length
 - all negative values
 - decimal result
-

12.9 Common Mistakes

Mistake 1: Integer division

Wrong:

```
return bestSum / k;
```

Correct:

```
return (double)bestSum / k;
```

Mistake 2: Initializing bestSum to 0

Wrong if all sums are negative.

12.10 Similar Problems

- Maximum Number of Vowels in a Substring of Given Length
 - Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold
 - Find All Anagrams in a String
-

13. Full Problem Walkthrough 2 — LeetCode 1456: Maximum Number of Vowels in a Substring of Given Length

13.1 Problem Summary

Given a string s and integer k , return maximum number of vowels in any substring of length k .

Example:

```
s = "abciidef"  
k = 3
```

Output:

```
3
```

Substring "iii" has 3 vowels.

13.2 How to Identify This Pattern

Problem says:

- substring
- length k
- maximum count

Fixed-size sliding window.

13.3 C++ Code

```
class Solution {
public:
    bool isVowel(char c) {
        return c == 'a' || c == 'e' || c == 'i' ||
               c == 'o' || c == 'u';
    }

    int maxVowels(string s, int k) {
        int count = 0;
        int best = 0;

        for (int right = 0; right < s.size(); right++) {
            if (isVowel(s[right])) {
                count++;
            }

            if (right >= k && isVowel(s[right - k])) {
                count--;
            }

            if (right >= k - 1) {
                best = max(best, count);
            }
        }

        return best;
    }
};
```

13.4 Dry Run

s = "abciidef"
k = 3

0	a	a	1	0

right	char	window	vowel count	best
1	b	ab	1	0
2	c	abc	1	1
3	i	bci	1	1
4	i	cii	2	2
5	i	iii	3	3

13.5 Complexity

Time: $O(n)$

Space: $O(1)$

13.6 Edge Cases

- no vowels
 - all vowels
 - $k = 1$
 - k equals string length
-

13.7 Similar Problems

- Maximum Average Subarray I
 - Minimum Recolors to Get K Consecutive Black Blocks
 - Find All Anagrams in a String
-

14. Full Problem Walkthrough 3 — LeetCode 3: Longest Substring Without Repeating Characters

14.1 Problem Summary

Given string s , return length of longest substring without repeating characters.

Example:

$s = \text{"abcabcbb"}$

Output:

3

Answer is "abc".

14.2 How to Identify This Pattern

Problem says:

- substring
- longest

- without repeating characters

This is variable-size sliding window with frequency.

14.3 Brute Force Idea

Try every substring and check if it has duplicates.

Time: $O(n^2)$ or worse.

14.4 Optimized Intuition

Maintain a window with no duplicates.

Add `s[right]`.

If it creates duplicate, shrink from left until valid again.

14.5 C++ Code — Frequency Array Version

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        vector<int> freq(128, 0);

        int left = 0;
        int best = 0;

        for (int right = 0; right < s.size(); right++) {
            freq[s[right]]++;

            while (freq[s[right]] > 1) {
                freq[s[left]]--;
                left++;
            }

            best = max(best, right - left + 1);
        }

        return best;
    }
};
```

14.6 Code Explanation

```
freq[s[right]]++;
```

Add current character.

```
while (freq[s[right]] > 1)
```

Only the newly added character can cause a duplicate.

```
freq[s[left]]--;  
left++;
```

Shrink until duplicate removed.

14.7 Dry Run

s = "abcabcbb"

0	a	add	0	a	1
1	b	add	0	ab	2
2	c	add	0	abc	3
3	a	duplicate, remove a	1	bca	3
4	b	duplicate, remove b	2	cab	3
5	c	duplicate, remove c	3	abc	3
6	b	duplicate, shrink	5	cb	3
7	b	duplicate, shrink	7	b	3

14.8 Alternative Last Seen Index Version

```
class Solution {  
public:  
    int lengthOfLongestSubstring(string s) {  
        vector<int> lastSeen(128, -1);  
  
        int left = 0;  
        int best = 0;  
  
        for (int right = 0; right < s.size(); right++) {  
            char c = s[right];  
  
            if (lastSeen[c] >= left) {  
                left = lastSeen[c] + 1;  
            }  
  
            lastSeen[c] = right;  
  
            best = max(best, right - left + 1);  
        }  
    }  
};
```

```
        return best;
    }
};
```

14.9 Important Trap in Last Seen Version

For:

```
s = "abba"
```

At last 'a', previous 'a' is outside the current window.

Do not move left backward.

That is why we check:

```
if (lastSeen[c] >= left)
```

14.10 Complexity

Time: $O(n)$

Space: $O(1)$, for ASCII.

14.11 Edge Cases

- empty string
- all same characters
- all unique characters
- spaces/symbols
- repeated character outside current window

14.12 Similar Problems

- Longest Substring with At Most Two Distinct Characters
- Fruit Into Baskets
- Longest Repeating Character Replacement
- Minimum Window Substring

15. Full Problem Walkthrough 4 — LeetCode 209: Minimum Size Subarray Sum

15.1 Problem Summary

Given positive integers `nums` and `target`, return minimal length of subarray whose sum is at least `target`.

Example:

```
target = 7
nums = {2, 3, 1, 2, 4, 3}
```

Output:

2

Best subarray:

{4, 3}

15.2 How to Identify This Pattern

Problem says:

- subarray
- minimum length
- sum at least target
- positive integers

This is shortest valid sliding window.

Positive integers make sum monotonic.

15.3 Brute Force Idea

Try every subarray.

Time: $O(n^2)$

15.4 Optimized Intuition

Expand right until sum is valid.

While valid, shrink from left and update best.

15.5 C++ Code

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int left = 0;
        int sum = 0;
        int best = INT_MAX;

        for (int right = 0; right < nums.size(); right++) {
            sum += nums[right];

            while (sum >= target) {
                best = min(best, right - left + 1);

                sum -= nums[left];
                left++;
            }
        }

        return best == INT_MAX ? 0 : best;
    }
};
```

```
}  
};
```

15.6 Dry Run

target = 7
nums = {2, 3, 1, 2, 4, 3}

0	2	2	no	none	INF
1	3	5	no	none	INF
2	1	6	no	none	INF
3	2	8	yes	len 4, remove 2	4
4	4	10	yes	len 4, remove 3; len 3, remove 1	3
5	3	9	yes	len 3, remove 2; len 2, remove 4	2

Return 2.

15.7 Complexity

Time: $O(n)$

Each pointer moves at most n times.

Space: $O(1)$

15.8 Edge Cases

- no valid subarray
- one element satisfies
- whole array needed
- target very small

15.9 Common Mistake

Using this with negative numbers.

If negatives are allowed, use prefix sum / monotonic deque depending on problem.

15.10 Similar Problems

- Minimum Window Substring
- Subarray Product Less Than K
- Shortest Subarray with Sum at Least K, needs monotonic deque

16. Full Problem Walkthrough 5 — LeetCode 567: Permutation in String

16.1 Problem Summary

Given strings s_1 and s_2 , return true if s_2 contains a permutation of s_1 .

Example:

```
s1 = "ab"
s2 = "eidbaooo"
```

Output:

```
true
```

Because "ba" is a permutation of "ab".

16.2 How to Identify This Pattern

Permutation of s_1 must have:

- same length as s_1
- same character frequencies

So use fixed-size frequency window.

16.3 Brute Force Idea

Check every substring of length $s_1.size()$ and sort/compare.

Time: $O(n * k \log k)$

16.4 Optimized Intuition

Maintain frequency of current window in s_2 .

Compare with frequency of s_1 .

Since alphabet size is 26, comparison is constant.

16.5 C++ Code

```
class Solution {
public:
    bool checkInclusion(string s1, string s2) {
        int k = s1.size();

        if (k > s2.size()) {
            return false;
        }

        vector<int> need(26, 0);
```

```

vector<int> window(26, 0);

for (char c : s1) {
    need[c - 'a']++;
}

for (int right = 0; right < s2.size(); right++) {
    window[s2[right] - 'a']++;

    if (right >= k) {
        window[s2[right - k] - 'a']--;
    }

    if (window == need) {
        return true;
    }
}

return false;
}
};

```

16.6 Dry Run

s1 = "ab"
s2 = "eidbaooo"

Need:

a:1, b:1

0	e	e	no
1	i	ei	no
2	d	id	no
3	b	db	no
4	a	ba	yes

Return true.

16.7 Complexity

Time: $O(26 * n)$, effectively $O(n)$

Space: $O(1)$

16.8 Edge Cases

- s1 longer than s2
 - repeated characters
 - exact same strings
 - no matching permutation
-

16.9 Common Mistake

Checking only if all characters exist.
Permutation requires exact counts.

16.10 Similar Problems

- Find All Anagrams in a String
 - Minimum Window Substring
 - Valid Anagram
-

17. Full Problem Walkthrough 6 — LeetCode 438: Find All Anagrams in a String

17.1 Problem Summary

Given strings *s* and *p*, return all starting indices of *p*'s anagrams in *s*.

Example:

```
s = "cbaebabacd"  
p = "abc"
```

Output:

```
{0, 6}
```

Because:

```
"cba"  
"bac"
```

are anagrams of "abc".

17.2 How to Identify This Pattern

Anagram window must have:

- fixed length `p.size()`
- same character counts as *p*

Use fixed-size frequency window.

17.3 C++ Code

```

class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        vector<int> result;

        int k = p.size();

        if (k > s.size()) {
            return result;
        }

        vector<int> need(26, 0);
        vector<int> window(26, 0);

        for (char c : p) {
            need[c - 'a']++;
        }

        for (int right = 0; right < s.size(); right++) {
            window[s[right] - 'a']++;

            if (right >= k) {
                window[s[right - k] - 'a']--;
            }

            if (window == need) {
                result.push_back(right - k + 1);
            }
        }

        return result;
    }
};

```

17.4 Dry Run

s = "cbaebabacd"
p = "abc"

2	cba	0	yes	{0}

right	window	start	anagram?	result
3	bae	1	no	{0}
4	aeb	2	no	{0}
5	eba	3	no	{0}
6	bab	4	no	{0}
7	aba	5	no	{0}
8	bac	6	yes	{0,6}

17.5 Complexity

Time: $O(26 * n)$, effectively $O(n)$

Space: $O(1)$

17.6 Edge Cases

- p longer than s
 - repeated characters
 - all same characters
 - overlapping anagrams
-

17.7 Common Mistake

Wrong start index.

If window length is k and ends at right, start is:

right - k + 1

17.8 Similar Problems

- Permutation in String
 - Minimum Window Substring
 - Substring with Concatenation of All Words
-

18. Full Problem Walkthrough 7 — LeetCode 76: Minimum Window Substring

18.1 Problem Summary

Given strings s and t, return the minimum window substring of s containing all characters of t, including duplicates.

Example:

```
s = "ADOBECODEBANC"
```

```
t = "ABC"
```

Output:

```
"BANC"
```

18.2 How to Identify This Pattern

Problem says:

- minimum window
- substring
- contains all characters
- duplicates matter

This is shortest valid sliding window with frequency counts.

18.3 Brute Force Idea

Try every substring and check whether it contains all characters of t.

Time: $O(n^2)$ or worse.

18.4 Optimized Intuition

Expand right until window satisfies all required characters.

Then shrink left while still valid to minimize length.

18.5 Important Variables

```
need[c]
```

How many of character c are needed.

```
window[c]
```

How many of character c are in current window.

```
required
```

Number of unique required characters.

```
formed
```

Number of unique required characters whose required count is satisfied.

18.6 C++ Code

```
class Solution {  
public:  
    string minWindow(string s, string t) {  
        if (t.size() > s.size()) {  
            return "";  
        }  
    }  
};
```

```
vector<int> need(128, 0);
vector<int> window(128, 0);

int required = 0;

for (char c : t) {
    if (need[c] == 0) {
        required++;
    }

    need[c]++;
}

int formed = 0;
int left = 0;

int bestLen = INT_MAX;
int bestStart = 0;

for (int right = 0; right < s.size(); right++) {
    char c = s[right];
    window[c]++;

    if (need[c] > 0 && window[c] == need[c]) {
        formed++;
    }

    while (formed == required) {
        int currentLen = right - left + 1;

        if (currentLen < bestLen) {
            bestLen = currentLen;
            bestStart = left;
        }

        char remove = s[left];
        window[remove]--;

        if (need[remove] > 0 && window[remove] < need[remove]) {
            formed--;
        }
    }
}
```

```

        left++;
    }
}

if (bestLen == INT_MAX) {
    return "";
}

return s.substr(bestStart, bestLen);
}
};

```

18.7 Code Explanation

required

Counts unique required characters.

For t = "AABC":

A:2, B:1, C:1

required = 3

formed

Increases when a character's required frequency is satisfied.

while (formed == required)

Window is valid, so try shrinking.

18.8 Dry Run Summary

s = "ADOBECODEBANC"

t = "ABC"

Important moments:

0	A	no	A satisfied
3	B	no	B satisfied
5	C	yes	best = ADOBEC
10	A	yes	shrink
12	C	yes	shrink to BANC

Final answer:

"BANC"

18.9 Complexity

Time: $O(n + m)$

Each pointer moves at most n times.

Space: $O(1)$, fixed ASCII array.

18.10 Edge Cases

- t longer than s
 - no valid window
 - duplicates in t
 - case sensitivity
 - one-character strings
 - all characters required
-

18.11 Common Mistakes

Mistake 1: Ignoring duplicates in t

If:

```
t = "AA"
```

Window must contain two As.

Mistake 2: Updating answer after removing left

Update while window is still valid before removing.

Mistake 3: Not decreasing formed

When removing a necessary character, update validity.

18.12 Similar Problems

- Permutation in String
 - Find All Anagrams
 - Minimum Window Subsequence, different pattern
 - Substring with Concatenation of All Words
-

19. Full Problem Walkthrough 8 — LeetCode 424: Longest Repeating Character Replacement

19.1 Problem Summary

Given uppercase string s and integer k , return length of longest substring that can become all same character after replacing at most k characters.

Example:

```
s = "AABABBA"
```

```
k = 1
```

Output:

```
4
```

One valid window:

"AABA"

Replace B with A.

19.2 How to Identify This Pattern

Problem asks:

- longest substring
- at most k changes
- character frequency inside window

Use variable sliding window.

19.3 Key Insight

For any window, best target character is the most frequent character in that window.

Replacements needed:

$\text{windowLength} - \text{maxFrequency}$

Window is valid if:

$\text{windowLength} - \text{maxFrequency} \leq k$

19.4 C++ Code

```
class Solution {
public:
    int characterReplacement(string s, int k) {
        vector<int> freq(26, 0);

        int left = 0;
        int best = 0;
        int maxFreq = 0;

        for (int right = 0; right < s.size(); right++) {
            freq[s[right] - 'A']++;

            maxFreq = max(maxFreq, freq[s[right] - 'A']);

            while ((right - left + 1) - maxFreq > k) {
                freq[s[left] - 'A']--;
                left++;
            }

            best = max(best, right - left + 1);
        }
    }
};
```

```
    return best;
}
};
```

19.5 Why maxFreq Does Not Decrease

This is subtle.

When shrinking, maxFreq may become stale.

But it still works because we only care about maximum length. A stale maxFreq may delay shrinking slightly, but it never causes an overestimated final answer in this pattern.

For beginners, you can recompute max frequency each time because alphabet size is only 26, but the above is standard.

19.6 Dry Run

s = "AABABBA"
k = 1

0	A	A	1	0	1	
1	A	AA	2	0	2	
2	B	AAB	2	1	3	
3	A	AABA	3	1	4	
4	B	AABAB	3	2	shrink	4

Answer: 4

19.7 Complexity

Time: $O(n)$

Space: $O(1)$

19.8 Edge Cases

- k = 0
- all same letters
- k large enough for whole string
- alternating characters

19.9 Common Mistakes

- Trying every target character manually
- Wrong formula for replacements
- Using lowercase indexing for uppercase string

19.10 Similar Problems

- Max Consecutive Ones III
 - Fruit Into Baskets
 - Longest Substring with At Most K Distinct Characters
-

20. Full Problem Walkthrough 9 — LeetCode 1004: Max Consecutive Ones III

20.1 Problem Summary

Given binary array and integer k, return maximum number of consecutive 1s if you can flip at most k zeroes.

Example:

```
nums = {1,1,1,0,0,0,1,1,1,0}
k = 2
```

Output:

```
6
```

20.2 How to Identify This Pattern

Problem asks:

- longest subarray
- at most k zeroes
- binary array

Use sliding window with zeroCount.

20.3 C++ Code

```
class Solution {
public:
    int longestOnes(vector<int>& nums, int k) {
        int left = 0;
        int zeroCount = 0;
        int best = 0;

        for (int right = 0; right < nums.size(); right++) {
            if (nums[right] == 0) {
                zeroCount++;
            }

            while (zeroCount > k) {
                if (nums[left] == 0) {
                    zeroCount--;
                }
                left++;
            }

            best = max(best, right - left + 1);
        }

        return best;
    }
};
```

```

        }

        left++;
    }

    best = max(best, right - left + 1);
}

return best;
}
};

```

20.4 Complexity

Time: $O(n)$

Space: $O(1)$

20.5 Edge Cases

- $k = 0$
- all ones
- all zeroes
- k greater than number of zeroes

20.6 Common Mistake

Resetting the window entirely when zero count exceeds k .

Correct approach:

Shrink gradually from left.

20.7 Similar Problems

- Longest Repeating Character Replacement
- Max Consecutive Ones II
- Longest Subarray of 1s After Deleting One Element

21. Full Problem Walkthrough 10 — LeetCode 904: Fruit Into Baskets

21.1 Problem Summary

Given fruits, return length of longest contiguous subarray containing at most two distinct fruit types.

Example:

fruits = {0, 1, 2, 2}

Output:

3

Best window:

{1, 2, 2}

21.2 How to Identify This Pattern

Problem says:

- longest contiguous segment
- at most two types

This is at most K distinct sliding window with k = 2.

21.3 C++ Code

```
class Solution {
public:
    int totalFruit(vector<int>& fruits) {
        unordered_map<int, int> freq;

        int left = 0;
        int best = 0;

        for (int right = 0; right < fruits.size(); right++) {
            freq[fruits[right]]++;

            while (freq.size() > 2) {
                freq[fruits[left]]--;

                if (freq[fruits[left]] == 0) {
                    freq.erase(fruits[left]);
                }

                left++;
            }

            best = max(best, right - left + 1);
        }

        return best;
    }
};
```

21.4 Dry Run

fruits = {0, 1, 2, 2}

0	0	{0:1}	yes	none	1
1	1	{0:1,1:1}	yes	none	2
2	2	{0:1,1:1,2:1}	no	remove 0	2
3	2	{1:1,2:2}	yes	none	3

Return 3.

21.5 Complexity

Time: $O(n)$ average

Space: $O(1)$, because at most 3 fruit types are stored during adjustment.

21.6 Common Mistake

Forgetting to erase zero-count keys.

If you do not erase, `freq.size()` stays too large.

21.7 Similar Problems

- Longest Substring with At Most Two Distinct Characters
- Longest Substring with At Most K Distinct Characters
- Subarrays with K Different Integers

22. Full Problem Walkthrough 11 — LeetCode 713: Subarray Product Less Than K

22.1 Problem Summary

Given positive integers `nums` and integer `k`, return number of contiguous subarrays where product is less than `k`.

Example:

`nums = {10, 5, 2, 6}`

`k = 100`

Output:

8

22.2 How to Identify This Pattern

Problem asks:

- count subarrays
- product less than `k`
- positive integers

Positive numbers allow sliding window.

22.3 Important Edge Case

If:

$k \leq 1$

No positive product can be less than k .
Return 0.

22.4 Core Counting Insight

After shrinking, if $[left \dots right]$ is valid, then all subarrays ending at $right$ and starting from left to right are valid.

Count:

$right - left + 1$

22.5 C++ Code

```
class Solution {
public:
    int numSubarrayProductLessThanK(vector<int>& nums, int k) {
        if (k <= 1) {
            return 0;
        }

        int left = 0;
        int answer = 0;
        long long product = 1;

        for (int right = 0; right < nums.size(); right++) {
            product *= nums[right];

            while (product >= k) {
                product /= nums[left];
                left++;
            }

            answer += right - left + 1;
        }

        return answer;
    }
};
```

22.6 Dry Run

nums = {10, 5, 2, 6}
k = 100

0	10	10	0	{10}	1
1	5	50	0	{5}, {10,5}	2
2	2	10	1	{2}, {5,2}	2
3	6	60	1	{6}, {2,6}, {5,2,6}	3

Total:

$1 + 2 + 2 + 3 = 8$

22.7 Complexity

Time: $O(n)$

Space: $O(1)$

22.8 Edge Cases

- $k \leq 1$
- one element
- all products valid
- product becomes large, use long long

22.9 Common Mistake

Counting only one subarray per right.

There can be many valid subarrays ending at each right.

22.10 Similar Problems

- Minimum Size Subarray Sum
- Count Number of Nice Subarrays
- Binary Subarrays With Sum
- Subarrays with K Different Integers

23. Full Problem Walkthrough 12 — LeetCode 1248: Count Number of Nice Subarrays

23.1 Problem Summary

Given array nums and integer k, return number of subarrays with exactly k odd numbers.

Example:

```
nums = {1, 1, 2, 1, 1}
```

```
k = 3
```

Output:

```
2
```

23.2 How to Identify This Pattern

Problem asks:

- count subarrays
- exactly k odd numbers

Exactly K is often solved using:

```
atMost(K) - atMost(K - 1)
```

23.3 Helper: Count Subarrays With At Most K Odd Numbers

```
int atMost(vector<int>& nums, int k) {  
    if (k < 0) {  
        return 0;  
    }  
  
    int left = 0;  
    int answer = 0;  
  
    for (int right = 0; right < nums.size(); right++) {  
        if (nums[right] % 2 == 1) {  
            k--;  
        }  
  
        while (k < 0) {  
            if (nums[left] % 2 == 1) {  
                k++;  
            }  
  
            left++;  
        }  
  
        answer += right - left + 1;  
    }  
}
```

```
    return answer;
}
```

23.4 Full C++ Code

```
class Solution {
public:
    int atMost(vector<int>& nums, int k) {
        if (k < 0) {
            return 0;
        }

        int left = 0;
        int answer = 0;

        for (int right = 0; right < nums.size(); right++) {
            if (nums[right] % 2 == 1) {
                k--;
            }

            while (k < 0) {
                if (nums[left] % 2 == 1) {
                    k++;
                }

                left++;
            }

            answer += right - left + 1;
        }

        return answer;
    }

    int numberOfSubarrays(vector<int>& nums, int k) {
        return atMost(nums, k) - atMost(nums, k - 1);
    }
};
```

23.5 Why This Works

Subarrays with at most 3 odd numbers include:

- exactly 0 odd
- exactly 1 odd

- exactly 2 odd
- exactly 3 odd

Subarrays with at most 2 odd numbers include:

- exactly 0 odd
- exactly 1 odd
- exactly 2 odd

Subtracting gives exactly 3 odd.

23.6 Complexity

Time: $O(n)$

Actually two passes, but $O(2n) = O(n)$.

Space: $O(1)$

23.7 Similar Problems

- Binary Subarrays With Sum
 - Subarrays with K Different Integers
 - Count Vowel Substrings
-

24. Full Problem Walkthrough 13 — LeetCode 930: Binary Subarrays With Sum

24.1 Problem Summary

Given binary array `nums` and integer `goal`, return number of subarrays with sum exactly `goal`.

Example:

```
nums = {1, 0, 1, 0, 1}
```

```
goal = 2
```

Output:

```
4
```

24.2 How to Identify This Pattern

Binary array has non-negative values.

Exact goal can be solved by:

```
atMost(goal) - atMost(goal - 1)
```

24.3 C++ Code

```
class Solution {  
public:  
    int atMost(vector<int>& nums, int goal) {
```

```
    if (goal < 0) {  
        return 0;  
    }  
  
    int left = 0;  
    int sum = 0;  
    int answer = 0;  
  
    for (int right = 0; right < nums.size(); right++) {  
        sum += nums[right];  
  
        while (sum > goal) {  
            sum -= nums[left];  
            left++;  
        }  
  
        answer += right - left + 1;  
    }  
  
    return answer;  
}  
  
int numSubarraysWithSum(vector<int>& nums, int goal) {  
    return atMost(nums, goal) - atMost(nums, goal - 1);  
}  
};
```

24.4 Complexity

Time: $O(n)$

Space: $O(1)$

24.5 Alternative Pattern

Can also solve with prefix sum + hash map.

Sliding window works here because array is binary/non-negative.

24.6 Similar Problems

- Count Number of Nice Subarrays
 - Subarrays with K Different Integers
 - Subarray Sum Equals K, prefix sum version for negatives
-

25. Full Problem Walkthrough 14 — LeetCode 992: Subarrays with K Different Integers

25.1 Problem Summary

Given array `nums` and integer `k`, return number of subarrays with exactly `k` distinct integers.

Example:

```
nums = {1, 2, 1, 2, 3}
k = 2
```

Output:

```
7
```

25.2 How to Identify This Pattern

Problem asks:

- count subarrays
- exactly `K` distinct

Use:

```
atMost(K) - atMost(K - 1)
```

with frequency map.

25.3 C++ Code

```
class Solution {
public:
    int atMost(vector<int>& nums, int k) {
        unordered_map<int, int> freq;

        int left = 0;
        int answer = 0;

        for (int right = 0; right < nums.size(); right++) {
            freq[nums[right]]++;

            while (freq.size() > k) {
                freq[nums[left]]--;
                if (freq[nums[left]] == 0) {
                    freq.erase(nums[left]);
                }
            }
        }
    }
};
```

```
        left++;
    }

    answer += right - left + 1;
}

return answer;
}

int subarraysWithKDistinct(vector<int>& nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}
};
```

25.4 Why Add $\text{right} - \text{left} + 1$?

After shrinking, the window $[\text{left} \dots \text{right}]$ has at most K distinct values.

Then every subarray ending at right and starting from any index between left and right is also valid.

25.5 Complexity

Time: $O(n)$ average

Space: $O(k)$

25.6 Common Mistakes

- Forgetting to erase zero-count keys
 - Trying to count exactly K directly
 - Confusing distinct count with frequency count
-

25.7 Similar Problems

- Fruit Into Baskets
 - Count Number of Nice Subarrays
 - Binary Subarrays With Sum
-

26. Full Problem Walkthrough 15 — LeetCode 1358: Number of Substrings Containing All Three Characters

26.1 Problem Summary

Given a string containing only a, b, and c, return number of substrings containing at least one of each.

Example:

```
s = "abcabc"
```

Output:

```
10
```

26.2 How to Identify This Pattern

Problem asks:

- count substrings
- contains all required characters
- small fixed alphabet

Use sliding window counting.

26.3 Intuition

Expand right until window contains all a, b, c.

When valid, every substring starting at left and ending at any index from right to end is valid.

So add:

```
s.size() - right
```

Then shrink left.

26.4 C++ Code

```
class Solution {
public:
    int numberOfSubstrings(string s) {
        vector<int> freq(3, 0);

        int left = 0;
        int answer = 0;

        for (int right = 0; right < s.size(); right++) {
            freq[s[right] - 'a']++;

            while (freq[0] > 0 && freq[1] > 0 && freq[2] > 0) {
                answer += s.size() - right;

                freq[s[left] - 'a']--;
                left++;
            }
        }
    }
}
```

```
        return answer;
    }
};
```

26.5 Dry Run Concept

For:

```
s = "abcabc"
```

When window first becomes "abc" at right = 2, all substrings starting at left = 0 and ending at indices 2,3,4,5 are valid.

Add:

```
6 - 2 = 4
```

Then shrink.

26.6 Complexity

Time: $O(n)$

Space: $O(1)$

26.7 Similar Problems

- Minimum Window Substring
- Count Vowel Substrings
- Subarrays with K Different Integers

27. Mini Problem 1 — LeetCode 1343: Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold

Pattern

Fixed-size window sum.

```
class Solution {
public:
    int numOfSubarrays(vector<int>& arr, int k, int threshold) {
        int targetSum = k * threshold;

        int windowSum = 0;
        int answer = 0;

        for (int right = 0; right < arr.size(); right++) {
```

```

        windowSum += arr[right];

        if (right >= k) {
            windowSum -= arr[right - k];
        }

        if (right >= k - 1 && windowSum >= targetSum) {
            answer++;
        }
    }

    return answer;
}
};

```

28. Mini Problem 2 — LeetCode 2379: Minimum Recolors to Get K Consecutive Black Blocks

Pattern

Fixed-size window count.

```

class Solution {
public:
    int minimumRecolors(string blocks, int k) {
        int whiteCount = 0;
        int answer = INT_MAX;

        for (int right = 0; right < blocks.size(); right++) {
            if (blocks[right] == 'W') {
                whiteCount++;
            }

            if (right >= k && blocks[right - k] == 'W') {
                whiteCount--;
            }

            if (right >= k - 1) {
                answer = min(answer, whiteCount);
            }
        }
    }
}

```

```
        return answer;
    }
};
```

Key idea:

In any length-k window, number of white blocks is number of recolors needed.

29. Mini Problem 3 — LeetCode 1208: Get Equal Substrings Within Budget

Pattern

Longest valid window with cost sum $\leq$ maxCost.

```
class Solution {
public:
    int equalSubstring(string s, string t, int maxCost) {
        int left = 0;
        int cost = 0;
        int best = 0;

        for (int right = 0; right < s.size(); right++) {
            cost += abs(s[right] - t[right]);

            while (cost > maxCost) {
                cost -= abs(s[left] - t[left]);
                left++;
            }

            best = max(best, right - left + 1);
        }

        return best;
    }
};
```

30. Mini Problem 4 — LeetCode 1493: Longest Subarray of 1s After Deleting One Element

Pattern

Longest window with at most one zero, then subtract one deletion.

```
class Solution {
public:
    int longestSubarray(vector<int>& nums) {
        int left = 0;
        int zeroCount = 0;
        int best = 0;

        for (int right = 0; right < nums.size(); right++) {
            if (nums[right] == 0) {
                zeroCount++;
            }

            while (zeroCount > 1) {
                if (nums[left] == 0) {
                    zeroCount--;
                }

                left++;
            }

            best = max(best, right - left);
        }

        return best;
    }
};
```

Why right - left?

Window length is right - left + 1, but you must delete one element.

So valid length after deletion is:

right - left

31. Mini Problem 5 — LeetCode

1658: Minimum Operations to Reduce X to Zero

Pattern

Transform problem.

Removing from left/right to sum to x is equivalent to keeping longest middle subarray with sum:

```
totalSum - x
class Solution {
public:
    int minOperations(vector<int>& nums, int x) {
        int total = 0;

        for (int num : nums) {
            total += num;
        }

        int target = total - x;

        if (target < 0) {
            return -1;
        }

        int left = 0;
        int sum = 0;
        int best = -1;

        for (int right = 0; right < nums.size(); right++) {
            sum += nums[right];

            while (sum > target && left <= right) {
                sum -= nums[left];
                left++;
            }

            if (sum == target) {
                best = max(best, right - left + 1);
            }
        }
    }
}
```

```
    if (best == -1) {  
        return -1;  
    }  
  
    return nums.size() - best;  
}  
};
```

Works because numbers are positive.

32. Common Edge Cases for Sliding Window

32.1 Empty Input

For longest length, return 0.

For minimum window string, return empty string.

For count, return 0.

32.2 $k = 0$

Important for:

- Max Consecutive Ones III
- Exactly K conversions
- At most K problems

For atMost(k), if $k < 0$, return 0.

32.3 k Larger Than Input

For fixed-size windows, often no full window exists unless constraints prevent it.

For replacement problems, whole string may be valid.

32.4 Repeated Characters

Important for:

- Longest substring without repeating
 - Character replacement
 - Minimum window
-

32.5 All Unique Characters

Longest no-repeat substring is whole string.

32.6 Negative Numbers

For sum-based sliding window, negative numbers often break logic.

Use prefix sum/hash map instead.

32.7 Duplicate Requirements

For:

```
t = "AABC"
```

Minimum window must contain two As.

32.8 Overflow

For product:

```
long long product = 1;
```

For large sums:

```
long long sum = 0;
```

33. Common Mistakes and Wrong Code

Mistake 1 — Updating Answer Before Window Is Valid

Wrong:

```
freq[s[right]]++;  
best = max(best, right - left + 1);  
  
while (invalid) {  
    shrink  
}
```

Correct:

```
add right  
  
while (invalid) {  
    shrink  
}  
  
update best
```

Mistake 2 — Using if Instead of while

Wrong:

```
if (invalid) {  
    shrink once;  
}
```

Sometimes one shrink is not enough.

Correct:

```
while (invalid) {  
    shrink;  
}
```

Mistake 3 — Forgetting to Remove Left From State

Wrong:

```
left++;
```

Correct:

```
freq[s[left]]--;  
left++;
```

or:

```
sum -= nums[left];  
left++;
```

Mistake 4 — Forgetting to Erase Zero Count

Wrong:

```
freq[x]--;
```

Correct:

```
freq[x]--;  
  
if (freq[x] == 0) {  
    freq.erase(x);  
}
```

Mistake 5 — Confusing Fixed and Variable Window

Fixed window:

```
if (size > k) shrink once
```

Variable window:

```
while (invalid) shrink
```

Mistake 6 — Using Sliding Window With Negative Exact Sum

For exact sum with negatives, use prefix sum.

Mistake 7 — Wrong Window Length

Correct:

```
right - left + 1
```

Wrong:

right - left

Except when intentionally deleting one element, like LC 1493.

34. Interview Explanation Templates

34.1 Fixed-Size Window

Since the window size is fixed at k , I maintain the sum/count/frequency of the current window. When the right pointer moves, I add the new element and remove the element that falls out. This lets me evaluate each window in $O(1)$.

34.2 Longest Valid Window

I expand the window with the right pointer. If the window becomes invalid, I move the left pointer until it becomes valid again. Then I update the maximum length.

34.3 Shortest Valid Window

I expand the window until it satisfies the condition. Once valid, I shrink from the left while preserving validity, updating the minimum length along the way.

34.4 Frequency Window

I maintain character counts for the current window and compare or validate them against required counts. Since each movement changes only one character, updates are efficient.

34.5 Count At Most

For each right endpoint, after making the window valid, every subarray ending at right and starting between left and right is valid, so I add $\text{right} - \text{left} + 1$.

35. Complexity Patterns

Fixed-size sum	$O(n)$	$O(1)$
Fixed-size frequency	$O(26n)$, effectively $O(n)$	$O(1)$
Longest no-repeat	$O(n)$	$O(1) / O(k)$
Minimum size subarray	$O(n)$	$O(1)$
Minimum window substring	$O(n + m)$	$O(1)$ ASCII
At most K distinct	$O(n)$ average	$O(k)$
Count valid windows	$O(n)$	$O(1) / O(k)$

Pattern	Time	Space
Exactly K via atMost	$O(n)$	$O(1) / O(k)$

36. Time-Boxed Practice Guidance

Fixed Window Easy

Target:

5–12 minutes

Examples:

- Maximum Average Subarray
- Maximum Vowels
- Minimum Recolors

Variable Window Medium

Target:

15–30 minutes

Examples:

- Longest Substring Without Repeating
- Minimum Size Subarray Sum
- Fruit Into Baskets
- Max Consecutive Ones III

Harder Frequency / Counting Windows

Target:

30–45 minutes

Examples:

- Minimum Window Substring
- Subarrays with K Different Integers
- Number of Substrings Containing All Three Characters

37. Expanded Practice Set

37.1 Must-Do Easy

- 1.1. Maximum Average Subarray I
- 2.2. Maximum Number of Vowels in a Substring of Given Length
- 3.3. Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold
- 4.4. Minimum Recolors to Get K Consecutive Black Blocks
- 5.5. Contains Duplicate II

37.2 Must-Do Medium

- 6.6. Longest Substring Without Repeating Characters
- 7.7. Minimum Size Subarray Sum

- 8.8. Permutation in String
 - 9.9. Find All Anagrams in a String
 - 10.10. Longest Repeating Character Replacement
 - 11.11. Max Consecutive Ones III
 - 12.12. Fruit Into Baskets
 - 13.13. Subarray Product Less Than K
 - 14.14. Count Number of Nice Subarrays
 - 15.15. Binary Subarrays With Sum
 - 16.16. Get Equal Substrings Within Budget
 - 17.17. Longest Subarray of 1s After Deleting One Element
 - 18.18. Minimum Operations to Reduce X to Zero
-

37.3 Strong Interview Practice

- 19.19. Minimum Window Substring
 - 20.20. Subarrays with K Different Integers
 - 21.21. Number of Substrings Containing All Three Characters
 - 22.22. Longest Substring with At Most K Distinct Characters
 - 23.23. Longest Substring with At Most Two Distinct Characters
 - 24.24. Replace the Substring for Balanced String
 - 25.25. Longest Substring with At Least K Repeating Characters, divide and conquer overlap
 - 26.26. Substring with Concatenation of All Words
 - 27.27. Frequency of the Most Frequent Element, sorting + window
 - 28.28. Maximize the Confusion of an Exam
-

37.4 Hard / Stretch

- 29.29. Minimum Window Substring
 - 30.30. Substring with Concatenation of All Words
 - 31.31. Sliding Window Maximum, monotonic deque
 - 32.32. Sliding Window Median, multiset/heaps
 - 33.33. Shortest Subarray with Sum at Least K, prefix + monotonic deque
 - 34.34. Subarrays with K Different Integers
-

38. Suggested Study Order

Step 1 — Fixed Window

- 19. Maximum Average Subarray I
- 20. Maximum Number of Vowels
- 21. Minimum Recolors
- 22. Number of Sub-arrays of Size K and Average $\geq$ Threshold

Goal:

- Learn add right, remove outgoing element.
-

Step 2 — Longest Valid Window

- 23. Longest Substring Without Repeating Characters
- 24. Max Consecutive Ones III
- 25. Fruit Into Baskets

26. Longest Repeating Character Replacement
- Goal:
- Learn expand until invalid, shrink until valid.
-

Step 3 — Shortest Valid Window

27. Minimum Size Subarray Sum
28. Minimum Window Substring
- Goal:
- Learn expand until valid, shrink while valid.
-

Step 4 — Frequency Matching

29. Permutation in String
30. Find All Anagrams in a String
- Goal:
- Learn exact count matching in fixed windows.
-

Step 5 — Counting Windows

31. Subarray Product Less Than K
32. Count Number of Nice Subarrays
33. Binary Subarrays With Sum
34. Subarrays with K Different Integers
35. Number of Substrings Containing All Three Characters
- Goal:
- Learn $\text{right} - \text{left} + 1$
 - Learn $\text{atMost}(K) - \text{atMost}(K - 1)$
-

39. Pattern Decision Table

“subarray of size k”	Fixed window	Add right, remove old
“substring length k”	Fixed frequency	Frequency arrays
“longest without/at most”	Longest valid	Shrink while invalid
“minimum/shortest containing”	Shortest valid	Shrink while valid
“contains permutation/anagram”	Fixed frequency	Compare counts
“at most k distinct”	Variable frequency	Map size $\leq k$
“count at most”	Counting window	answer $+= \text{right} - \text{left} + 1$
“exactly k”	At-most conversion	$\text{atMost}(k) - \text{atMost}(k - 1)$

Problem Clue	Sliding Window Type	Template
		1)
“positive sum/product”	Sum/product window	Expand/shrink
“negative exact sum”	Not sliding window	Prefix sum + map

40. Final Cheat Sheet

Fixed Window

```

int left = 0;
int windowSum = 0;
int best = INT_MIN;

for (int right = 0; right < nums.size(); right++) {
    windowSum += nums[right];

    if (right - left + 1 > k) {
        windowSum -= nums[left];
        left++;
    }

    if (right - left + 1 == k) {
        best = max(best, windowSum);
    }
}

```

Longest Valid Window

```

int left = 0;
int best = 0;

for (int right = 0; right < nums.size(); right++) {
    // add nums[right]

    while (/* invalid */) {
        // remove nums[left]
        left++;
    }
}

```

```
    best = max(best, right - left + 1);  
}
```

Shortest Valid Window

```
int left = 0;  
int best = INT_MAX;  
  
for (int right = 0; right < nums.size(); right++) {  
    // add nums[right]  
  
    while (/* valid */) {  
        best = min(best, right - left + 1);  
  
        // remove nums[left]  
        left++;  
    }  
}
```

At Most K Distinct

```
unordered_map<int, int> freq;  
  
int left = 0;  
int best = 0;  
  
for (int right = 0; right < nums.size(); right++) {  
    freq[nums[right]]++;  
  
    while (freq.size() > k) {  
        freq[nums[left]]--;  
  
        if (freq[nums[left]] == 0) {  
            freq.erase(nums[left]);  
        }  
  
        left++;  
    }  
  
    best = max(best, right - left + 1);  
}
```

Count Valid Windows

```
int left = 0;
int answer = 0;

for (int right = 0; right < nums.size(); right++) {
    // add nums[right]

    while (/* invalid */) {
        // remove nums[left]
        left++;
    }

    answer += right - left + 1;
}
```

Exactly K

```
int exactlyK(vector<int>& nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}
```

41. What to Master Before Chapter 5

Before moving to Prefix Sum, make sure you can:

- Identify contiguous subarray/substring problems
 - Distinguish fixed-size and variable-size windows
 - Maintain window sum
 - Maintain window product
 - Maintain window frequency
 - Expand right correctly
 - Shrink left correctly
 - Know when to use while
 - Update answer at the correct time
 - Use $\text{right} - \text{left} + 1$
 - Count valid windows
 - Use $\text{atMost}(K) - \text{atMost}(K - 1)$
 - Erase zero-count keys from maps
 - Know when sliding window fails with negative numbers
-

42. Core Takeaway

Sliding Window is about maintaining a valid contiguous segment efficiently.

The main questions are:

- 36. What does the window represent?
- 37. What state do I maintain?
- 38. What makes the window invalid or valid?
- 39. When do I shrink?
- 40. When do I update the answer?

Most sliding window solutions follow this structure:

```
left = 0
```

```
for right in range:
```

```
    add right element
```

```
    while window is invalid or valid depending on goal:
```

```
        update/remove left
```

```
        left++
```

```
    update answer
```

If you can define the window state and validity condition, sliding window problems become much easier.
